APES (Advanced Parallel Execution System) Platform Information

📌 Platform Overview

APES is a platform for **distributed multi-agent orchestration**. Its main goal is to take a high-level task, decompose it into subtasks, and execute them efficiently in parallel using multiple AI agents and a **DAG (Directed Acyclic Graph)** based scheduler.

It focuses heavily on **Reinforcement-based agent selection** and **Memory-driven optimization**, meaning it learns which agents perform best over time and routes tasks accordingly. The design is explicitly "simulation-first", allowing the architecture to behave predictably and scalably even before connecting to external LLMs.

🛠️ Technology Stack

The platform is extraordinarily lightweight and relies on standard modern web technologies, specifically designed with **zero external dependencies**.

- **Core Runtime: Pure Node.js (Version 20+)**.
- **Module System:** ECMAScript Modules (`type: "module"` in `package.json`).
- **Internal Paradigm:** Event-driven architecture, Worker Pools, and Mathematical DAG (Directed Acyclic Graph) Scheduling.
- **Future Integration:** The architecture outlines upcoming integrations for **LLM Providers** and **Vector Databases** (specifically `pgvector` / `Supabase` for long-term memory embeddings).

⚙️ How It Works: Step-by-Step (The 10-Stage Pipeline)

When you execute a task via the APES CLI (e.g., `apes "build a REST API with authentication"`), it follows a rigorous **10-Stage Cognitive Pipeline**:

Phase 1: Preparation

1. Parse: The Interface Layer's CLI Parser captures your intent and creates a secure session context, handling any permission/side-effect gates upfront. **2. Classify (Intent):** The platform categorizes what type of problem is being solved. **3. Decompose:** It breaks down the high-level prompt into a series of smaller, discrete subtasks. **4. Score (Complexity):** Each breakdown is assigned a complexity score based on dependency weight and risk factor:

- **Simple (Score 0-3):** Routes to 1 or 2 agents.
- **Medium (Score 4-7):** Routes to 3-5 agents simultaneously.
- **Complex (Score 8+):** Activates DAG wave execution.

Phase 2: Action

5. Allocate: The Dynamic Spawner selects the best agents for the job. Instead of randomly choosing, it picks agents based on past confidence metrics. Agents are drawn from a Registry of **11 specific agents divided into 6 clusters** (Research, Coding, DevOps, UI/UX, Analysis, Eval). **6. Execute (DAG Scheduler):** Subtasks are executed. Instead of doing them one by one, a DAG scheduler maps out dependencies and executes independent tasks in **waves** using a bounded Worker Pool (max 8 workers).

- *Example:* Wave 1 runs Task [A]. Wave 2 runs Tasks [B, C] in parallel after A finishes.

Phase 3: Review & Evolution

7. Evaluate: The system evaluates the results returned by the agent workers. **8. Aggregate:** Individual subtask results are stitched back together into a cohesive final outcome. **9. Learn (Learning System):** The orchestrator updates the system via Reinforcement Scoring.

- *Example:* If an agent finishes faster than average, its confidence score goes up (`+0.02`). If it fails, confidence goes down (`-0.05`). This continuously sharpens agent selection routing for your next commands. **10. Output:** The final result is served back to the user through the terminal CLI renderer.

🧠 Memory Subsystems

A massive component of how the platform "learns" during this process is its **4-Layer Memory Architecture**:

1. **Session Memory:** Caches the immediate context of the current request.
2. **Performance Memory:** Keeps persistent track of agent metrics and success trends.
3. **Skill Evolution:** Stores heuristics and abstract patterns the system figures out along the way.
4. **Vector Memory (Future):** Slated for LLM-based RAG (Retrieval-Augmented Generation) embeddings.